

NPC DIALOGUE MIDDLEWARE

Software Engineering for Artificial Intelligence

`npc-dialogue-middleware`

Studenti

Claudio Buono - NF22500051

Gianpio Silvestri - NF22500083

Docente

Fabio Palomba

ANNO ACCADEMICO 2025/2026

Indice

1 Panoramica **3**

1.1 Introduzione . . . 3

1.2 Obiettivi del Progetto . . . 3

1.3 Destinatari . . . 5

2 Sistema proposto **6**

2.1 Requisiti Funzionali . . . 6

2.2 Requisiti Non Funzionali . . . 8

2.3 Architettura del Sistema . . . 9

2.3.1 Vista di deployment . . . 9

2.3.2 Workflow principale . . . 10

3 Componenti del Sistema **11**

3.1 L'Orchestrator . . . 11

3.1.1 Modalità di Generazione del Dialogo . . . 11

3.2 Gestione della Memoria . . . 12

3.3 Modelli di Contesto . . . 12

3.3.1 `GameContext` . . . 13

3.3.2 `NPCContext` . . . 14

3.3.3 `Dialogue` e `Quest` . . . 15

3.3.4 `Talkativeness` . . . 16

3.4 Pre-processing . . . 16

3.5 Costruzione del Contratto . . . 17

3.5.1 Input e output . . . 17

3.5.2 Composizione del Prompt . . . 18

3.6 Routing Dinamico LLM . . . 19

3.6.1 Profiling . . . 19

3.6.2 Complexity Analyzer . . . 20

3.6.3 Selezione finale . . . 21

3.7 Refiner . . . 22

3.7.1 Judger . . . 22

3.7.2 Healer . . . 23

3.8 Monitoraggio del Sistema 24
3.8.1 State Manager 24
3.8.2 Telemetry 24
3.8.3 Logging 25
3.9 Configurabilità 26
3.9.1 Categorie di parametri configurabili 26
3.9.2 Modalità di configurazione 27

4 Prompt Engineering **28**
4.1 System-Aware Prompting 28
4.2 Tecniche di Prompt Engineering 29
4.2.1 Role Prompting 29
4.2.2 Field Grounding 30
4.2.3 Directive Prompting 30
4.3 Regole esplicite e vincoli operativi 31
4.3.1 Regole di contenuto e coerenza narrativa 31
4.3.2 Controllo delle allucinazioni 32

5 Fairness **33**
5.1 Controllo statico basato su parole vietate 33
5.2 Controllo in fase di pre-processing 33
5.3 Controllo in fase di post-processing 34
5.4 Regole di fairness nel prompt 34
5.5 Configurabilità dei controlli 35

6 Risultati **36**
6.1 Valutazioni empiriche 36
6.2 Testing Metamorfico 37
6.2.1 Relazioni metamorfiche considerate 37
6.2.2 Limiti del testing metamorfico 38
6.3 Esempio di Dialogo 38
6.4 Conclusioni e Sviluppi Futuri 40

1 Panoramica

1.1 Introduzione

I videogiochi moderni ricercano livelli di immersione sempre più elevati, e le interazioni tra giocatore e personaggi non giocanti (NPC) sono uno degli aspetti in cui questa ricerca è più evidente. I tradizionali alberi di dialogo garantiscono un controllo narrativo totale, ma per loro natura offrono un numero finito di scambi predefiniti. L'uso dei Large Language Models permette di affiancare a questi strumenti dialoghi generati dinamicamente, che si adattano alle azioni del giocatore, allo stato del mondo e alle caratteristiche dell'NPC.
Questo progetto propone lo sviluppo di un middleware progettato per assistere gli sviluppatori nella creazione e nel prototyping di dialoghi dinamici. L'obiettivo non è sostituire il processo creativo degli autori, bensì fornire uno strumento che acceleri la produzione di contenuti mantenendo la coerenza narrativa.
Il middleware integra meccanismi di controllo volti a mitigare bias, contenuti inappropriati e possibili allucinazioni dei modelli linguistici, promuovendo un impiego etico e sicuro degli LLM all'interno del processo di sviluppo videoludico.

All'interno del documento sono stati inclusi appositi riquadri di approfondimento che illustrano il razionale di ciascuna scelta progettuale, formattati come segue:

> *Abbiamo preferito questa impostazione per valorizzare il razionale delle nostre decisioni durante la lettura, oltre che per ragioni di resa visiva.*

1.2 Obiettivi del Progetto

L'obiettivo di questo progetto è quello di sviluppare è un middleware che renda semplice, sicuro ed efficiente l'uso dei Large Language Models nei videogiochi, ponendosi come livello intermedio tra il videogioco e i modelli linguistici che generano i dialoghi. In questo modo chi sviluppa non deve occuparsi degli aspetti tecnici dell'integrazione con gli LLM e può concentrarsi sul design e sulla narrazione del gioco.

Il progetto si articola sui seguenti obiettivi specifici:

- **Generazione Contestuale:** Produrre dialoghi in tempo reale che siano coerenti con lo stato attuale del mondo di gioco e con la progressione narrativa, in modo che ogni dialogo risulti credibile e coerente con quanto accaduto fino a quel momento.
- **Astrazione e Standardizzazione:** Definire un'interfaccia universale per lo scambio di dati tra *game engine* e modelli linguistici, eliminando le barriere di integrazione tra tecnologie eterogenee e rendendo il middleware indipendente dallo specifico *game engine*.
- **Configurabilità:** Rendere il sistema adattabile a contesti di gioco diversi tramite impostazioni esterne al codice, così da sostituire i modelli e regolare il comportamento del middleware in modo semplice e veloce, senza modificare il videogioco.
- **Ottimizzazione delle Performance:** Contenere latenza e costi, sia computazionali sia economici, tramite strategie di instradamento delle richieste verso i modelli più adatti e una gestione efficiente del flusso di richieste.
- **Affidabilità e Allineamento Narrativo:** Aumentare il controllo semantico sui testi generati combinando *Prompt Engineering* e sistemi di validazione, per assicurare che le risposte rispettino il tono, i personaggi e le regole narrative definite dagli sviluppatori.
- **Sicurezza ed Etica:** Filtrare automaticamente contenuti inappropriati, offensivi, discriminatori o frutto di allucinazioni.
- **Osservabilità:** Fornire strumenti di logging e telemetria per monitorare prestazioni, consumo di risorse e qualità delle risposte, sia in fase di test sia in produzione, così da individuare problemi e margini di miglioramento.

1.3 Destinatari

Il middleware è pensato per le diverse figure professionali del *Game Development*, che condividono un obiettivo comune: integrare l'IA generativa nei propri giochi in modo semplice, senza dover gestire la complessità tecnica dei modelli linguistici. In particolare, i principali beneficiari individuati sono:

- **Sviluppatori Indipendenti:** Singoli o piccoli team che, con budget e risorse limitati, necessitano di uno strumento pronto all'uso per prototipare rapidamente meccaniche basate su dialoghi dinamici.
- **Studi Strutturati (AA/AAA):** Realtà industriali interessate a integrare NPC avanzati in mondi di gioco complessi, beneficiando di un sistema centralizzato per sicurezza, telemetria e performance.
- **Narrative Designer e Writer:** Professionisti che desiderano velocizzare la scrittura di dialoghi secondari, mantenendo il pieno controllo creativo sulla trama e sulla caratterizzazione dei personaggi.

2 Sistema proposto

2.1 Requisiti Funzionali

Il sistema dovrà soddisfare i seguenti requisiti funzionali:

- [**RF1**] **Generazione del contratto narrativo.** Il sistema deve accettare in input il contesto del gioco ed i dati dell'NPC, generando un *contratto narrativo* che definisce le informazioni necessarie alla generazione del dialogo.
- [**RF2**] **Generazione del dialogo dell'NPC.** Il sistema deve produrre un dialogo per l'NPC utilizzando esclusivamente le informazioni contenute nel contratto narrativo precedentemente generato. I dialoghi devono risultare coerenti con il ruolo, la personalità, il contesto e gli obiettivi del personaggio, mantenendo uno stile uniforme e contestualmente appropriato.
- [**RF3**] **Generazione delle possibili risposte del giocatore.** A partire dalla linea di dialogo generata per l'NPC e dal contratto narrativo, il sistema deve creare un insieme di possibili risposte del giocatore. Le opzioni proposte devono essere semanticamente coerenti con il contesto della conversazione, offrire differenti possibilità di interazione e rispettare i vincoli definiti dal contratto.
- [**RF4**] **Gestione della risposta del giocatore.** Il sistema deve accettare in input la risposta del giocatore al dialogo precedente e utilizzarla come base per la generazione della replica successiva.
- [**RF5**] **Instradamento dinamico dei modelli.** Il sistema deve analizzare automaticamente la complessità della richiesta e instradare il task verso il modello linguistico più appropriato.

- [**RF6**] **Modalità di generazione.** Il sistema deve supportare due modalità di generazione dei contenuti, selezionabile per ogni richiesta:
  - **Modalità normale:** il contenuto viene generato per intero e restituito solo dopo aver superato la pipeline di validazione completa. È adatta ai casi in cui la risposta può essere preparata in anticipo, senza vincoli di tempo stringenti.
  - **Modalità stream:** il contenuto viene restituito in modo incrementale, chunk dopo chunk, applicando i soli controlli statici sui chunk prodotti. È pensata per contenuti che richiedono bassa latenza.
- [**RF7**] **Validazione dell'output.** Il sistema deve sottoporre il contenuto generato a una fase di validazione, volta a identificare e mitigare contenuti tossici, discriminatori, offensivi, affetti da bias o caratterizzati da possibili allucinazioni dell'LLM. La profondità della validazione dipende dalla modalità di generazione: in modalità normale il sistema applica la pipeline completa, con valutazione e raffinamento iterativo; in modalità stream applica esclusivamente controlli statici sui chunk prodotti.
- [**RF8**] **Gestione del fallimento della validazione.** Se in modalità normale il contenuto non supera la validazione entro il numero massimo di iterazioni di raffinamento consentito, il sistema non deve restituire il contenuto non valido, ma segnalare esplicitamente l'errore al client, che può gestirlo con un contenuto di ripiego.
- [**RF9**] **Tracciamento, logging e telemetria.** Il sistema deve raccogliere e registrare informazioni dettagliate sul funzionamento dei vari componenti. Tali informazioni devono supportare attività di monitoraggio, debugging e analisi delle prestazioni.

2.2 Requisiti Non Funzionali

Il sistema dovrà soddisfare i seguenti requisiti non funzionali:

- **[RNF1] Agnosticismo tecnologico.** Il sistema deve essere progettato in modo indipendente dalle tecnologie proprietarie dei diversi game engine. L'architettura deve consentire l'integrazione con motori di gioco differenti mediante interfacce ben definite, riducendo le dipendenze dalla piattaforma di destinazione e favorendo la portabilità del sistema.
- **[RNF2] Prestazioni.** Il sistema deve garantire tempi di risposta compatibili con un'interazione in tempo reale. In modalità stream la latenza percepita deve essere minimizzata; in modalità normale sono tollerati tempi maggiori, purché la generazione avvenga in anticipo rispetto al momento in cui il contenuto serve.
- **[RNF3] Coerenza narrativa.** I contenuti generati devono risultare coerenti con il contesto narrativo del videogioco, rispettando la personalità, il ruolo, gli obiettivi e l'allineamento dell'NPC, nonché gli eventi e le regole definite nel contesto del gioco. Il sistema deve evitare la generazione di dialoghi che risultino incoerenti o in contrasto con l'ambientazione.
- **[RNF4] Sicurezza ed equità.** Il sistema deve integrare meccanismi di controllo in grado di individuare e mitigare contenuti offensivi, discriminatori, tossici o caratterizzati da bias sociali. Tali meccanismi devono operare prevalentemente mediante tecniche di prompt engineering e guardrail.
- **[RNF5] Configurabilità.** Il sistema deve consentire di adattarne il comportamento alle necessità dello sviluppatore tramite la configurazione dei principali parametri operativi. Le modifiche alla configurazione non devono richiedere interventi sul codice del videogioco.

2.3 Architettura del Sistema

2.3.1 Vista di deployment

Il middleware comunica con il *game engine* esclusivamente tramite HTTP e può essere eseguito sia sullo stesso dispositivo del videogioco sia su un server remoto, senza alcuna modifica all'interfaccia esposta all'engine.

> *Dato il requisito di engine-agnosticism, HTTP ci è sembrata la scelta più sensata: è supportato da qualsiasi engine, senza plugin specifici, e permette di passare da un'esecuzione locale a una remota semplicemente cambiando l'indirizzo IP della macchina.*

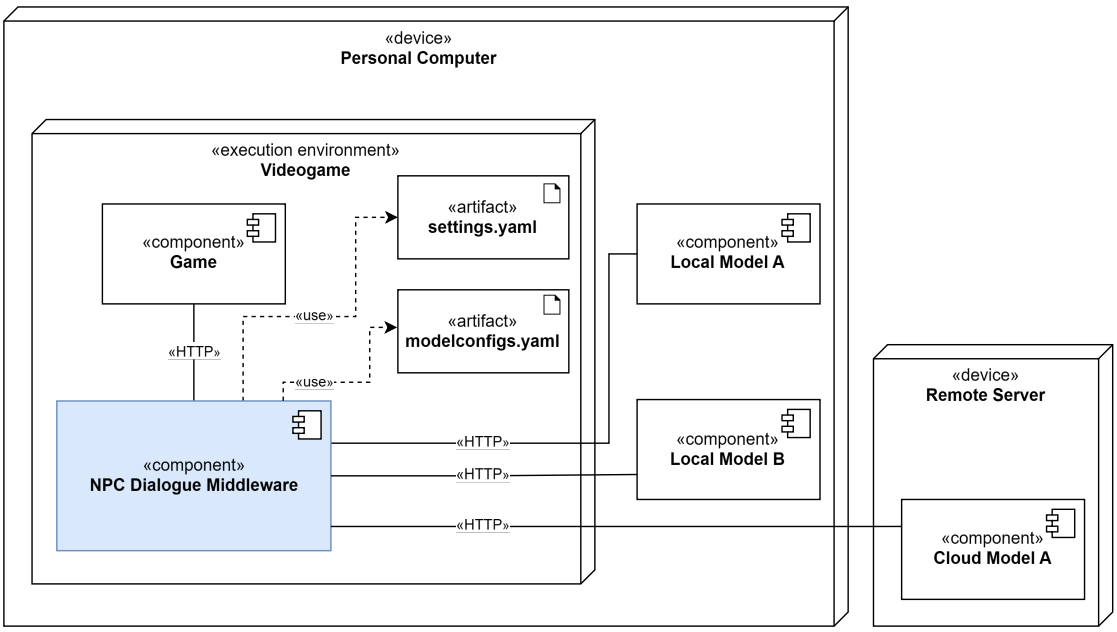


Figura 2.1: Deployment diagram del sistema in locale

Per semplificare l'integrazione, il progetto include un sistema di build che produce l'eseguibile del middleware insieme alle risorse necessarie al suo funzionamento. In questo modo il middleware può essere distribuito insieme al gioco e avviato come processo in background durante l'esecuzione.

La configurazione locale è la più semplice da adottare e, con modelli eseguiti in locale, non dipende dalla rete. La posizione remota resta comunque possibile ed è particolarmente utile per i giochi esclusivamente online, dove un unico servizio può essere condiviso da più giocatori.

2.3.2 Workflow principale

La richiesta di generazione di un dialogo attraversa tre macro-fasi: il *pre-processing* dei dati provenienti dal *game engine*, la generazione vera e propria e la validazione del dialogo generato. Le tre fasi sono coordinate dall'**Orchestrator**. Il flusso è il seguente:

1. **Pre-processing**: valida e normalizza i dati ricevuti dall'engine intercettando già in ingresso eventuali termini vietati.
2. **Contract Builder**: costruisce il contratto per il modello linguistico, composto dal prompt e dallo schema della risposta atteso, che impone un formato strutturato rendendo l'output prevedibile e verificabile.
3. **LLM Router**: seleziona provider e modello più appropriati in base alla complessità della richiesta e alla configurazione del sistema.
4. **Dialogue Generator**: invia il contratto al modello selezionato e raccoglie la risposta strutturata.
5. **Validazione e Output**: sottopone la risposta ai controlli previsti dalla modalità di generazione richiesta e, una volta validata, la impacchetta e la invia al *game engine*.

3 Componenti del Sistema

3.1 L'Orchestrator

Dopo aver validato i dati ottenuti dal *game engine*, questi vengono passati all'Orchestrator, che funge da punto d'accesso unificato e da coordinatore logico per tutte le operazioni di generazione dei dialoghi e delle opzioni di gioco.
L'Orchestrator disaccoppia la logica di gioco dai componenti interni di basso livello, gestendo in modo centralizzato il ciclo di vita delle richieste, la transizione degli stati del sistema, la selezione dei modelli e l'applicazione delle pipeline di validazione.

3.1.1 Modalità di Generazione del Dialogo

L'architettura supporta due modalità distinte di generazione del testo, entrambe gestite e coordinate dall'Orchestrator. La scelta tra le due riflette un compromesso progettuale: la modalità normale privilegia l'**affidabilità** dell'output, mentre quella in streaming privilegia la **reattività** percepita dall'utente.

1. **Modalità Normale:** Gestisce il flusso end-to-end in blocco e offre il livello di controllo più elevato. Questa modalità integra un pipeline di post-processing, che comprende il ciclo di raffinamento iterativo del `Refiner` (con valutazione del `Judger` e correzione da parte dell'`Healer`) e i controlli statici.

   Il dialogo viene consegnato al *game engine* esclusivamente al termine di tutti i controlli, e solo se questi hanno esito positivo.

2. **Modalità Streaming:** Garantisce la minima latenza con il *game engine*. Il flusso testuale generato dal modello viene trasmesso in tempo reale e lavora per *chunk*: ciascun frammento viene restituito al *game engine* non appena disponibile, senza attendere il completamento della generazione. Per preservare la bassa latenza, i controlli sono volutamente ridotti: il ciclo di raffinamento non è previsto, ed è supportata la sola modalità di **censura**.

3.2 Gestione della Memoria

Il sistema di **Gestione della Memoria** ha il compito di mantenere il contesto della conversazione *in corso* tra il giocatore e l'NPC. Il middleware non archivia le conversazioni passate: conserva in memoria soltanto l'ultima interazione attiva, e questo è sufficiente per capire se il giocatore sta proseguendo un dialogo già avviato o ne sta iniziando uno nuovo.

> *La persistenza a lungo termine dei dialoghi, se necessaria, resta responsabilità del game engine, che può tradurre gli esiti delle conversazioni in modifiche al contesto dell'NPC.*

Avere a disposizione lo storico della conversazione corrente è importante per due motivi. Da un lato preserva la coerenza narrativa all'interno del dialogo, perché il modello può richiamare quanto già detto ed evitare ripetizioni e contraddizioni. Dall'altro fornisce ai moduli di raffinamento del dialogo (`Refiner`, `Judger` e `Healer`) il contesto necessario per valutare e correggere il dialogo in modo appropriato.

3.3 Modelli di Contesto

Per garantire un disaccoppiamento robusto tra il *game engine* e il middleware, le informazioni scambiate non vengono gestite come dati grezzi o strutture generiche, bensì formalizzate all'interno di modelli di dati fortemente tipizzati.
I modelli di dati principali utilizzati per la generazione dei dialoghi sono:

- `GameContext`: definisce lo stato del mondo di gioco. Raccoglie le informazioni ambientali e di scenario necessarie a garantire che il modello mantenga una percezione coerente del mondo circostante.
- `NPCContext`: definisce l'NPC, con i suoi attributi individuali, lo stato interno e gli obiettivi, per guidarne comportamento e interazioni.

Una volta ricevuti dal *game engine*, questi modelli accompagnano la richiesta lungo l'intero workflow di elaborazione: vengono validati e normalizzati in fase di preprocessing, forniscono il materiale per la costruzione del prompt, orientano la scelta

del modello da parte del router e costituiscono il riferimento rispetto al quale viene verificata la coerenza del dialogo generato. Ogni modulo opera così su una struttura dati uniforme e prevedibile, senza dover interpretare formati differenti.

> *Per la definizione di questi modelli abbiamo scelto Pydantic, che già garantisce la correttezza* tipologica *dei dati in ingresso al confine dell'API (ad esempio, che un campo numerico non contenga una stringa).*

3.3.1 `GameContext`

Il modello `GameContext` definisce l'ambientazione globale e la cornice narrativa in cui opera l'NPC. Questa struttura dati fornisce al modello una guida affinché le risposte generate rispettino la coerenza storica, stilistica e atmosferica del gioco. La conoscenza di questo contesto evita che l'NPC generi dialoghi fuori luogo rispetto allo stato del mondo.

Campo	**Descrizione**
`epoch`	Epoca storica o ambientazione del gioco (es. Medievale, Sci-Fi, Cyberpunk). *Obbligatorio.*
`environment`	Descrizione generale del mondo, atmosfera, clima o stile architettonico. *Obbligatorio.*
`world_state`	Situazione globale attuale o eventi rilevanti di dominio pubblico nel mondo di gioco. *Obbligatorio.*
`main_character_description`	Aspetto fisico del protagonista. I tratti di personalità vanno inclusi solo se ampiamente noti nel mondo di gioco. *Opzionale.*

Tabella 3.1: Campi del modello `GameContext`.

3.3.2 `NPCContext`

Il modello `NPCContext` definisce il profilo del singolo NPC all'interno della generazione del dialogo. Questa struttura dati fornisce al modello una guida affinché il dialogo rispetti il tono, l'atteggiamento e le motivazioni del personaggio.

Campo	**Descrizione**
`name`	Nome completo, titolo o alias dell'NPC. *Obbligatorio.*
`age`	Età dell'NPC. *Obbligatorio.*
`personality`	Profilo psicologico e tratti comportamentali dell'NPC. *Obbligatorio.*
`context`	Stato immediato dell'NPC: posizione attuale, ora del giorno e attività in corso. *Obbligatorio.*
`intent`	Scopo narrativo che guida la conversazione (`Dialogue` o `Quest`). *Obbligatorio.*
`talkativeness`	Livello di verbosità dell'NPC. *Obbligatorio.*
`main_character_relation`	Atteggiamento, posizione e relazione dell'NPC nei confronti del protagonista. *Obbligatorio.*
`recent_plot`	Eventi narrativi recenti che riguardano direttamente questo NPC. *Opzionale.*
`visual_description`	Aspetto fisico, abbigliamento, equipaggiamento ed espressioni visibili. *Opzionale.*
`backstory`	Storia, esperienze passate e motivazioni profonde dell'NPC. *Opzionale.*
`language`	Lingue parlate dall'NPC. Se assente, si applica la configurazione di lingua predefinita di sistema. *Opzionale.*

Tabella 3.2: Campi del modello `NPCContext`.

3.3.3 `Dialogue` e `Quest`

`Dialogue` è il blocco base, dedicato a tutte le conversazioni con l'NPC:

Campo	**Descrizione**
`type`	Identifica questo blocco come un dialogo semplice (`Dialogue`).
`must_use_expression`	Informazione che l'NPC deve necessariamente comunicare. *Opzionale.*
`more_info`	Dettagli contestuali per guidare la generazione e le scelte di dialogo del giocatore. *Opzionale.*
`number_of_options`	Il numero di opzioni di risposta da generare per il giocatore (default: `0`). *Opzionale.*

Tabella 3.3: Campi della struttura `Dialogue`.

`Quest` aggiunge campi per la definizione di una quest che l'NPC dovrà offrire nel dialogo.

Campo	**Descrizione**
`type`	Identifica questo blocco come un dialogo composto (`Quest`).
`objective`	L'obiettivo principale da raggiungere. *Obbligatorio.*
`name`	Titolo della quest. *Opzionale.*
`description`	Descrizione della quest. *Opzionale.*
`location`	Luogo dove poter svolgere la quest. *Opzionale.*
`reward`	Ricompensa ottenuta al completamento della quest. *Opzionale.*
`has_choice`	Indica se il giocatore può scegliere di accettare o rifiutare la quest (default: `False`). *Opzionale.*

Tabella 3.4: Campi della struttura `Quest`.

3.3.4 `Talkativeness`

`Talkativeness` è un parametro che definisce il livello di verbosità dell'NPC, ovvero quanto è lungo e articolato il dialogo.

Valore	**Descrizione**
`VERY_LOW`	Estremamente conciso; risponde con pochissime parole o monosillabi.
`LOW`	Riservato e schivo; fornisce risposte brevi.
`AVERAGE`	Bilanciato; mantiene un flusso di conversazione naturale e proporzionato.
`HIGH`	Espansivo e logorroico; tende ad allungarsi nelle spiegazioni e a condividere dettagli.
`VERY_HIGH`	Molto verboso e travolgente; aggiunge molti aneddoti o digressioni.

Tabella 3.5: Possibili valori di `Talkativeness`.

3.4 Pre-processing

Il modulo di *pre-processing* si occupa della validazione sintattica e della normalizzazione dei dati ricevuti dal middleware, prima che questi vengano utilizzati per la generazione dei dialoghi dell'NPC.
Alcuni dei controlli sintattici effettuati:

- vincoli di lunghezza minima/massima sui campi testuali;
- controllo di campi vuoti o composti unicamente da spazi bianchi;
- range numerici plausibili;

Qualora la fase di pre-processing individui errori sintattici o termini vietati, la richiesta viene interrotta prima di raggiungere il modulo di generazione dei dialoghi e al client viene restituita una risposta di errore contenente la descrizione del problema riscontrato.

3.5 Costruzione del Contratto

Il modulo `ContractBuilder` rappresenta il primo passo operativo dell'Orchestrator dopo la validazione dei dati. Il suo compito è tradurre lo stato del gioco in una richiesta non ambigua destinata al modello linguistico. Tale richiesta prende il nome di `Contract`, perché stabilisce un vero e proprio accordo tra il sistema e l'LLM: da una parte ciò che il modello deve sapere e rispettare, dall'altra il formato esatto in cui deve rispondere.

3.5.1 Input e output

Il `ContractBuilder` riceve in ingresso un `GameContext` e un `NPCContext`, entrambi già validati e normalizzati nelle fasi precedenti. Poiché i dati sono garantiti coerenti, il builder non deve occuparsi di controlli difensivi sull'input e può concentrarsi esclusivamente sulla composizione del prompt. In uscita produce un `Contract`, composto da tre elementi:

- **System prompt:** definisce il ruolo del modello, le regole di comportamento e i vincoli generali. Contiene le istruzioni stabili, come il tono narrativo, la coerenza con il personaggio interpretato, i limiti sui contenuti e le indicazioni sul formato di risposta, e incorpora le informazioni caratterizzanti dell'NPC (personalità, ruolo, loquacità) tratte dall'`NPCContext`.
- **User prompt:** contiene la parte dinamica della richiesta, cioè la situazione corrente di gioco: lo stato del mondo, la cronologia recente della conversazione, l'ultima azione del giocatore e quanto serve a generare la risposta del turno in corso. Viene ricostruito a ogni richiesta a partire dal `GameContext`.
- **Schema di output JSON:** descrive la struttura che la risposta del modello deve rispettare, con campi, tipi e vincoli. Includerlo nel contratto, anziché affidarsi a istruzioni testuali generiche, riduce il rischio di risposte malformate.

Una volta costruito, il `Contract` viene restituito all'Orchestrator, che lo utilizza per i passaggi successivi della pipeline: la selezione del modello tramite il router e la generazione vera e propria.
In questo modo ogni NPC parte da un contesto pulito, senza ereditare battute di dialogo che non ha mai pronunciato né ascoltato, e le informazioni di conversazioni diverse non si mescolano. Inoltre la memoria resta di dimensioni contenute, perché contiene sempre una sola conversazione.

3.5.2 Composizione del Prompt

Il contratto testuale viene assemblato separando nettamente il prompt di sistema da quello utente. Il processo ha natura **condizionale**: ogni blocco di testo o regola è incluso solo se rilevante per la richiesta corrente (ad esempio in base alla presenza di un intento specifico o di una cronologia pregressa), evitando di sovraccaricare il contesto con informazioni superflue. Lo stesso principio vale per lo schema di output JSON, esteso dinamicamente solo quando il task lo richiede.
I controlli basati sul prompt hanno però un limite, soprattutto in modalità *streaming*, dove la generazione continua può far degradare progressivamente il rispetto dei vincoli testuali. Per questo ai controlli dinamici si affiancano validazioni e filtri statici, applicati a valle dal middleware prima di restituire il risultato al gioco.

> *Abbiamo posto particolare attenzione a ridurre la ridondanza nei prompt, concentrando la capacità di elaborazione dell'LLM sui soli vincoli e dati essenziali.*

3.6 Routing Dinamico LLM

Il sistema di *Routing Dinamico* seleziona automaticamente il modello linguistico più appropriato per ogni richiesta, bilanciando qualità, costo computazionale e latenza. Ciò richiede di stimare sia la complessità della richiesta sia le prestazioni dei modelli disponibili. Il processo si articola in tre fasi:

- il *Profiling*, che classifica i modelli disponibili;
- il *Complexity Analyzer*, che stima la difficoltà di ogni richiesta;
- la *selezione finale*, che li mette in corrispondenza.

Quando è configurato un solo modello, la selezione viene bypassata (niente profiling né analisi della complessità) e tutte le richieste vanno direttamente a quel modello. Il routing entra in gioco solo in presenza di più modelli, tra cui il sistema sceglie in base alla complessità della richiesta e alle prestazioni stimate.

3.6.1 Profiling

Per instradare una richiesta occorre sapere quanto è capace e quanto è veloce ciascun modello. Il sistema deve funzionare con modelli arbitrari, anche sconosciuti a priori, e il criterio di classificazione deve quindi essere indipendente dallo specifico modello e produrre risultati verificabili.

> *Si è scelto il profiling automatico perché è l'unico criterio misurabile e indipendente dallo specifico modello. Numero di parametri, euristiche sul nome e autovalutazione dichiarata sono imprecisi, non standardizzati o non verificabili, mentre un altro modello IA come classificatore aggiungerebbe latenza e un margine d'errore poco compatibile con un processo prevedibile.*

Il sistema combina **profiling automatico** e **autovalutazione configurabile**. Il primo fornisce una valutazione oggettiva basata sulle prestazioni osservate; la seconda permette allo sviluppatore, quando dispone di informazioni affidabili sul modello, di indicarne il livello tramite la configurazione, evitando il profiling. Poiché le misure dirette sono preferibili a quelle dichiarate, il profiling è abilitato

per impostazione predefinita: in assenza di un'autovalutazione, il sistema profila automaticamente i modelli.
Il profiling misura il comportamento di ciascun modello nell'ambiente di esecuzione effettivo, tramite due metriche:

- **Time To First Token (TTFT)**: tempo tra l'invio della richiesta e la ricezione del primo token; descrive la latenza iniziale percepita dall'utente.
- **Throughput**: numero di token prodotti al secondo; descrive la velocità di generazione una volta avviata la risposta.

Le misure si ottengono inviando a ciascun modello un insieme ridotto di richieste di prova, ripetute più volte e aggregate per limitare l'effetto delle fluttuazioni di rete e di carico. Il costo del profiling, in termini di tempo e token consumati, viene sostenuto una sola volta: l'esecuzione avviene all'avvio e i risultati sono riutilizzati nelle richieste successive. In base ai valori ottenuti, i modelli vengono ordinati e assegnati a livelli di capacità, da associare ai livelli di complessità prodotti dal *Complexity Analyzer*.

3.6.2 Complexity Analyzer

Il *Complexity Analyzer* stima la complessità di ogni richiesta di generazione di un dialogo, per instradarla verso un modello più o meno capace (e costoso): evita di impiegare un modello potente per interazioni semplici e riserva risorse adeguate a quelle impegnative. L'analisi combina tre fattori in un unico punteggio normalizzato tra 0 e 1 tramite somma pesata:

- **Tipo di intento**: una battuta di dialogo è meno complessa della definizione di una quest, che richiede coerenza narrativa su più elementi e maggiore sforzo creativo. È il segnale più affidabile della difficoltà di generazione infatti gli è stato assegnato un peso maggiore.
- **Lunghezza del contesto**: quantità di testo in input; contesti più estesi richiedono maggiore capacità di comprensione e sintesi.

- **Ricchezza dell'intento**: numero di dettagli opzionali specificati (informazioni aggiuntive, opzioni di dialogo, oppure obiettivo, ricompensa, luogo e descrizione per una quest). Più campi sono popolati, più vincoli la generazione deve rispettare.

Il punteggio viene tradotto in tre livelli discreti:

- **Bassa (LOW)**: interazioni brevi e generiche, senza dettagli rilevanti.
- **Media (MEDIUM)**: contesto più articolato o alcuni dettagli opzionali.
- **Alta (HIGH)**: interazioni estese, ricche di dettagli o narrativamente significative.

> *Per la stima della complessità non si è usato un LLM, per lo stesso motivo visto per il routing: l'inferenza aggiungerebbe un collo di bottiglia all'esecuzione. In un generatore di dialoghi per videogiochi la velocità di risposta è cruciale, almeno per la modalità streaming.*

3.6.3 Selezione finale

La selezione finale mette in corrispondenza il livello di complessità della richiesta con il livello di capacità dei modelli profilati: le richieste **LOW** sono instradate verso i modelli più veloci ed economici, quelle **MEDIUM** verso i modelli intermedi e quelle **HIGH** verso i più capaci. Se più modelli appartengono allo stesso livello, si sceglie quello con le prestazioni misurate migliori (TTFT più basso, a parità di livello, throughput più alto). Se nessun modello copre il livello richiesto, si ricorre al livello più vicino disponibile.

3.7 Refiner

Il `Refiner` è il modulo che verifica e corregge i dialoghi generati prima che raggiungano il *game engine*. Un LLM può infatti produrre testo incoerente o non conforme ai vincoli del contratto; il `Refiner` interviene su questi casi coordinando due moduli: il `Judger`, che valuta il testo, e l'`Healer`, che lo corregge.
Il ciclo di valutazione e correzione procede finché il dialogo non risulta valido o finché non si esaurisce un budget configurabile. Come budget è stato scelto il numero massimo di iterazioni, una soluzione semplice da controllare e sufficiente a garantire un buon compromesso tra qualità del risultato e tempo di risposta. Se al termine del budget il dialogo non risulta ancora conforme, il `Refiner` non restituisce il contenuto non valido, ma segnala l'errore al *game engine*, che può gestirlo con un contenuto di ripiego.

> *Per i compiti di `Judger` e `Healer` abbiamo reso possibile riutilizzare lo stesso modello impiegato per generare il dialogo. Questo perché il rischio di self-preference bias è limitato: al `Judger` viene richiesto di valutare un singolo dialogo rispetto a un contesto e a vincoli espliciti, e all'`Healer` di correggere problemi già individuati. In nessuno dei due casi c'è un confronto tra alternative, e il giudizio poggia su criteri oggettivi e verificabili.*

3.7.1 Judger

Il `Judger` combina due tipi di verifica. I controlli semantici sono affidati a un LLM, a cui viene sottoposto un insieme di domande a risposta sì/no, costruito a partire dal contesto dell'NPC e dalle impostazioni di fairness e filtro del linguaggio. Ogni risposta negativa diventa una segnalazione e deve essere accompagnata da una motivazione; se questa è troppo breve, il giudice viene interrogato di nuovo, fino a un massimo di tre tentativi. I controlli statici, invece, sono deterministici e vengono eseguiti direttamente sul dialogo, senza costi aggiuntivi in termini di token.

Controlli Statici

I controlli statici verificano le regole che non richiedono un giudizio semantico, e restituiscono le segnalazioni che si aggiungono a quelle prodotte dall'LLM:

- **Espressione obbligatoria:** la frase che l'NPC deve pronunciare, se prevista dall'intento narrativo, è presente nel dialogo.
- **Numero di opzioni:** le opzioni proposte al giocatore sono tante quante richieste.
- **Accettazione e rifiuto:** se la scelta è prevista, sono presenti entrambe le risposte di accettazione e di rifiuto.
- **Indicazioni sceniche:** il testo non contiene azioni o didascalie, che il motore di gioco non saprebbe interpretare.
- **Lingua:** il dialogo è scritto interamente nella lingua configurata. Ogni sezione dell'output viene analizzata separatamente e segnalata se contiene almeno una parola in un'altra lingua; il nome dell'NPC viene ignorato, per evitare falsi positivi sui nomi propri.

3.7.2 Healer

Il modulo **Healer** corregge il dialogo segnalato dal `Judger` come problematico, non conforme o frutto di allucinazioni, evitando di scartarlo e rigenerarlo da zero. Per ogni intervento, l'`Healer` riceve il dialogo da correggere, il contesto di gioco e dell'NPC e l'elenco delle segnalazioni del `Judger`. Con questi elementi il `Contract Builder` costruisce un contratto dedicato, che chiede al modello di riscrivere il dialogo risolvendo i problemi indicati.
Il contratto viene inviato all'LLM con una temperatura propria, configurabile separatamente da quelle degli altri ruoli. L'`Healer` non verifica da sé l'esito della correzione, compito che spetta al `Judger` nell'iterazione successiva del ciclo coordinato dal `Refiner`.

3.8 Monitoraggio del Sistema

Il corretto funzionamento del middleware e la sincronizzazione dello stato di gioco richiedono un'infrastruttura di supporto trasversale, incaricata di monitorare il ciclo di vita delle richieste, tracciare le metriche operative e registrare dettagliatamente gli eventi di sistema.

3.8.1 State Manager

Il componente `StateManager` governa le transizioni di stato del middleware durante l'intero ciclo di elaborazione, garantendo un controllo rigoroso e deterministico sul flusso operativo. Il comportamento del sistema è modellato attraverso una macchina a stati finiti che definisce esplicitamente le diverse fasi operative:

- `IDLE`: Stato di riposo. Il sistema è inattivo e pronto a ricevere nuove richieste di generazione o configurazione.
- `STARTING`: Fase di inizializzazione e avvio dei componenti del middleware.
- `SETTING_CONTEXT`: Fase in cui il middleware riceve, valida e memorizza il *game context*.
- `GENERATING`: Fase attiva di generazione del dialogo.

Questa gestione centralizzata previene conflitti operativi e assicura che le chiamate asincrone rispettino rigorosamente i vincoli logici e temporali stabiliti dall'architettura di gioco.

3.8.2 Telemetry

Il modulo di telemetria monitora in modo trasparente le prestazioni del middleware e dei modelli linguistici sottostanti, senza interferire con l'esperienza di gioco. Per ciascun modello raccoglie le seguenti metriche:

- **Time To First Token (TTFT)**: latenza iniziale prima della ricezione del primo token;

- **Durata complessiva** della generazione;
- **Throughput**: numero di token generati al secondo;
- **Memoria VRAM** allocata dai diversi modelli.

Le statistiche cumulative per modello vengono aggiornate in modo incrementale e salvate su file tramite scritture atomiche, per evitare corruzioni in caso di arresti anomali. La telemetria ha uno scopo esclusivamente di monitoraggio: i dati non influenzano direttamente il routing o altre decisioni del sistema, ma permettono agli sviluppatori di analizzare le prestazioni dei modelli e individuare eventuali colli di bottiglia.

3.8.3 Logging

Il sistema di logging assicura la tracciabilità delle operazioni del middleware attraverso diversi livelli di severità, dal livello `INFO`, destinato al monitoraggio generale, fino al livello `DEBUG`, che consente l'ispezione dei prompt, dei contesti di gioco e della cronologia dei dialoghi. Il flusso di esecuzione viene registrato in forma strutturata su file di log giornalieri.
I log consentono di analizzare come varia la qualità dei dialoghi generati al modificare i prompt o la configurazione del sistema, e facilitano l'individuazione di comportamenti anomali. Permettono inoltre a chi integra il middleware di monitorarne il funzionamento e di verificare la conformità degli output prodotti.

3.9 Configurabilità

Uno degli obiettivi del progetto è rendere il middleware adattabile a contesti di gioco diversi senza modificare il codice. Per questo i principali parametri di comportamento non sono valori fissi nella logica applicativa, ma impostazioni configurabili.

3.9.1 Categorie di parametri configurabili

Le impostazioni esposte dal middleware si raggruppano in base all'aspetto che regolano:

- **Lingua di output**: determina la lingua del dialogo dell'NPC, coerentemente con la regola linguistica inclusa nel prompt di sistema.
- **Modelli linguistici disponibili**: elenca i modelli utilizzabili, locali o via API remota, indicando per ciascuno endpoint, dimensione massima del contesto, eventuale chiave di accesso e livello di potenza dichiarato, su cui si basa l'instradamento. Aggiungere, rimuovere o sostituire un modello richiede quindi la sola modifica di questo elenco.
- **Parametri del modello linguistico**: regolano il comportamento generativo. Il principale è la temperatura, che controlla la variabilità delle risposte e può essere impostata separatamente per ciascun ruolo del modello (generazione, valutazione e correzione).
- **Controlli di sicurezza e fairness**: il filtro sulle parole vietate e le regole di fairness nel prompt si abilitano indipendentemente, e si può personalizzare il testo che sostituisce le parole vietate.
- **Raffinamento dei dialoghi**: abilita o disabilita il ciclo di valutazione e correzione in modalità normale e ne fissa il numero massimo di iterazioni, regolando il compromesso tra qualità dei controlli e tempo di risposta.
- **Comportamento narrativo**: regola la struttura del dialogo generato, come il numero di opzioni di risposta proposte al giocatore.

3.9.2 Modalità di configurazione

Le impostazioni possono essere definite in due modi complementari.
Il primo è tramite file YAML esterni, separati dal codice: uno per le impostazioni generali e uno per l'elenco dei modelli. Definiscono la configurazione di default all'avvio, senza modificare o ricompilare il middleware.
Il secondo è tramite chiamate dirette che permettono di leggere o modificare le impostazioni a runtime. Questo è utile in due scenari: consentire al Game Engine di adattare il comportamento del middleware allo stato del gioco (ad esempio disattivando il filtro sulle parole vietate in contesti dove il design prevede un linguaggio più esplicito) e supportare le sessioni di sperimentazione, in cui la configurazione cambia rapidamente tra un test e l'altro senza riavviare il sistema.
Per preservare la coerenza dei dialoghi, alcune impostazioni non sono modificabili durante una generazione in corso: una richiesta di cambio lingua, ad esempio, viene rifiutata finché il middleware sta elaborando un dialogo.

4 Prompt Engineering

Dato che il dialogo di un NPC dipende principalmente dal prompt in ingresso fornito al modello, la sua progettazione ha avuto un ruolo centrale nel lavoro. Le sole istruzioni nel prompt non offrono però alcuna garanzia sul comportamento del modello: per questo motivo il prompt è affiancato da un'architettura che implementa meccanismi di controllo e validazione dell'output. Per lo stesso motivo, i prompt non sono stati trattati come semplice testo scritto una volta per tutte, ma come veri e propri artefatti software: componenti modulari, riutilizzabili e parametrizzabili, tenuti separati dalla logica applicativa del middleware.
In questa sezione vengono descritte le principali tecniche di prompt engineering adottate nella progettazione dei prompt, e la metodologia con cui questi vengono combinati per formare il prompt finale inviato al modello.

4.1 System-Aware Prompting

Utilizzando un approccio *System-Aware*, il prompt non si basa su testi statici, bensì su un'architettura dinamica assemblata in tempo reale a partire dallo stato corrente del mondo di gioco e dai parametri di configurazione dell'NPC. Per garantire che il modello generi dialoghi perfettamente sintonizzati sulla specifica situazione narrativa, le informazioni di contesto vengono iniettate nel prompt in modo rigorosamente strutturato, racchiudendole all'interno di tag XML-like espliciti.

```
<world_context>
    - Epoch: {epoch}
    - Environment: {environment}
    - Current world state: {world_state}
    - Main character appearance: {main_character_description}
</world_context>
```

L'adozione di questi delimitatori assolve a una duplice funzione fondamentale:

1. **Delimitazione semantica e parsing:** Aiuta il modello a distinguere nettamente tra le istruzioni di sistema (fisse e imperative) e i dati dinamici di

gioco, riducendo le ambiguità e guidando l'*attention mechanism* dell'LLM verso le informazioni rilevanti.

2. **Mitigazione della manipolazione (Safety):** I tag isolano i dati di gioco impedendo che eventuali stringhe arbitrarie inserite dall'utente o dal mondo di gioco vengano interpretate impropriamente come comandi o istruzioni di sistema, prevenendo fenomeni di *prompt injection* indiretta.

> *Sebbene il middleware permetta di configurare dinamicamente la lingua di generazione dei dialoghi, l'intera struttura dei prompt è stata scritta in inglese. Questa decisione architetturale sfrutta la superiore capacità di comprensione e aderenza ai vincoli dimostrata dagli LLM sulle istruzioni in lingua inglese, mantenendo al contempo la generazione dell'output completamente localizzabile.*

4.2 Tecniche di Prompt Engineering

4.2.1 Role Prompting

Il primo blocco che compare in ogni prompt di sistema assegna esplicitamente un ruolo al modello. Questa tecnica, nota come *role prompting*, viene utilizzata per ancorare fin da subito lo stile e il registro delle risposte del modello al compito richiesto, riducendo il rischio che il modello produca risposte generiche o fuori contesto. Il ruolo cambia in base allo stadio della pipeline e ha una propria persona (autore dei dialoghi, revisore, riparatore), con compiti e vincoli distinti.
Di seguito un esempio applicato, estratto dai prompt del sistema:

```
# ROLE
You are a narrative designer generating dialogue for NPCs
(non-player characters) in a videogame.
You write the NPC's spoken lines and the replies the player can
choose from.
You are never a narrator: everything you write is direct speech in
first person, with no descriptions of actions.
```

4.2.2 Field Grounding

Molti dei parametri che descrivono l'NPC, come la personalità o il livello di loquacità, sono concetti astratti che possono essere interpretati dal modello in modi diversi se forniti senza ulteriori spiegazioni. Per ridurre questa ambiguità, ogni campo utilizzato nel prompt è accompagnato da una breve spiegazione testuale del suo significato e del suo utilizzo previsto, prima ancora che venga fornito il valore effettivo.

```
# FIELD GUIDANCE
- Personality: the NPC's emotional state, attitude, and moral
compass.
- Context: the NPC's current situation, objectives, and immediate
environment.
- Relationship: initial level of trust, warmth, or hostility toward
the main character.
- Language / Dialect: vocabulary, tone, slang, or structural quirks
of their speech.
- Recent Events: immediate past occurrences that influence their
current mood or focus.
- Additional information: extra facts about the NPC, usable as
given.
```

4.2.3 Directive Prompting

Un'ulteriore tecnica adottata riguarda il modo in cui sono formulati i vincoli all'interno del prompt. Abbiamo osservato che l'impiego di direttive imperative e marcate, ottenute tramite parole chiave in maiuscolo come `MANDATORY`, `DO NOT` e `ONLY`, favorisce una maggiore aderenza del modello alle istruzioni fornite. Il loro uso è quindi riservato ai vincoli che non devono essere violati, mentre le indicazioni di stile o di preferenza sono lasciate in forma descrittiva, così da non diluire l'effetto delle prime.

Di seguito un estratto delle regole sulle quest:

```
# QUEST RULES
- The quest objective is MANDATORY content: the NPC must state it
explicitly, in their own voice, without omitting, generalizing, or
weakening it.
- DO NOT add quest details beyond those provided.
```

> *I prompt non includono dialoghi di esempio (few-shot). I modelli più piccoli tendono ad affidarsi troppo all'esempio quando vanno in difficoltà, riproducendone lessico e struttura invece di adattarsi all'NPC. Il comportamento atteso è quindi descritto solo tramite regole e definizioni dei campi.*

4.3 Regole esplicite e vincoli operativi

Oltre alle informazioni di contesto fornite dinamicamente, l'architettura dei prompt adotta un insieme rigoroso di regole generali che il modello deve rispettare in ogni caso, indipendentemente dal tipo di NPC o dalla specifica situazione narrativa. Tali direttive sono suddivise in categorie funzionali distinte per garantire un controllo granulare sul comportamento del generatore.

4.3.1 Regole di contenuto e coerenza narrativa

Comprendono vincoli positivi e negativi finalizzati alla stabilità della narrazione:

- **Vincoli positivi (*Mandatory Content*):** impongono che i contenuti obbligatori, come una Quest da assegnare, siano espressi dal punto di vista dell'NPC.
- **Vincoli negativi:** vietano gli elementi che anticipano impropriamente la scelta del giocatore, ad esempio menzionare l'accettazione o il rifiuto di una quest nelle opzioni di dialogo neutre, e l'introduzione di stereotipi legati a genere, etnia o background sociale nella caratterizzazione dei personaggi.

4.3.2 Controllo delle allucinazioni

Una categoria specifica di regole è dedicata alla prevenzione delle allucinazioni e alla salvaguardia del contesto di gioco. Il prompt stabilisce una distinzione netta tra due tipologie di output:

- **Dettagli stilistici consentiti:** Il modello è autorizzato ad aggiungere tocchi stilistici che non introducano nuovi fatti (es. intercalari, esitazioni, ritmo, inflessioni dialettali o sfumature emotive coerenti con la personalità dell'NPC).
- **Contenuti fattuali vietati:** Viene posto un divieto tassativo sull'invenzione di fatti (nomi di persone o luoghi, eventi passati, relazioni o dettagli di trama non presenti nei tag di input). In assenza di informazioni specifiche, il modello è istruito a mantenersi sul generico o sul vago anziché inventare.

Questa distinzione consente di preservare la naturalezza espressiva e la ricchezza stilistica del dialogo, blindando al contempo la coerenza logica e cronologica dell'universo di gioco contro incongruenze e informazioni non autorizzate.

5 Fairness

La fairness non è affidata a un unico punto del sistema, ma a più controlli distribuiti lungo il percorso di un dialogo. Prima di costruire il prompt si filtrano gli input, il prompt stesso contiene regole esplicite per il modello e, una volta generata la risposta, un ultimo controllo ne verifica il contenuto. Così un contenuto problematico ha più occasioni di essere fermato, e un singolo controllo che fallisce o viene aggirato non basta a compromettere il sistema.

5.1 Controllo statico basato su parole vietate

Il primo livello di controllo è di tipo statico: non richiede l'intervento del modello linguistico, ma si basa sul confronto diretto del testo con un insieme predefinito di termini non appropriati (come insulti, termini discriminatori o parole legate a stereotipi). Per strutturare questa base di conoscenza in modo rigoroso, è stato impiegato **HurtLex** [1], un lessico multilingue specificamente progettato per identificare termini offensivi, denigratori e discriminatori. La lista derivata da questa risorsa è mantenuta tramite un file di configurazione esterno, facilmente aggiornabile senza dover modificare il codice del middleware.
Per rendere efficiente la ricerca di queste parole all'interno di un testo, il lessico viene utilizzato per costruire un automa tramite l'algoritmo di **Aho-Corasick** [2]. Questo algoritmo permette di cercare contemporaneamente tutte le parole vietate all'interno di una stringa in un'unica scansione del testo, con un costo computazionale lineare rispetto alla lunghezza del testo analizzato. Questo aspetto è particolarmente importante nel nostro contesto, poiché il controllo deve poter essere eseguito più volte durante l'esecuzione (sia in input che in output) senza introdurre latenza significativa.

5.2 Controllo in fase di pre-processing

Il primo utilizzo dell'automa avviene nella fase di pre-processing, ovvero prima ancora che vengano costruiti i prompt da inviare al modello. In questa fase vengono analizzati sia il contesto di gioco sia il contesto specifico dell'NPC.

Lo scopo di questo controllo è duplice: da un lato evitare che contenuti già inappropriati presenti nell'input vengano propagati al modello linguistico; dall'altro prevenire che tali contenuti influenzino la generazione del dialogo, dato che il modello tende a essere condizionato dal contesto fornito nel prompt.

5.3 Controllo in fase di post-processing

Il secondo utilizzo dell'automa avviene a valle della generazione, sul testo prodotto dal modello. Il controllo viene applicato in modo diverso a seconda della modalità di generazione richiesta:

- **Modalità streaming**: il testo viene controllato progressivamente, chunk dopo chunk. Il comportamento in caso di rilevamento dipende dal *profanity filter*. Se il filtro è attivo, la generazione prosegue senza interruzioni e la parola vietata viene sostituita da una parola di censura definita nella configurazione, così da non mostrare mai il contenuto inappropriato al giocatore.
- **Modalità normale**: il controllo viene eseguito sul testo completo. Se è attivo il *fairness filter*, durante la fase di *refinement* vengono poste al modello ulteriori domande sulla fairness del testo generato, così da verificarne e, se necessario, correggerne il contenuto prima della restituzione.

5.4 Regole di fairness nel prompt

Oltre ai controlli statici descritti in precedenza, che agiscono a livello testuale, il sistema integra un livello di controllo più lieve direttamente nel System Prompt inviato al modello. In particolare, vengono aggiunte al prompt delle regole esplicite di fairness, come mostrato di seguito:

```
# FAIRNESS RULES
- Avoid stereotypes related to the NPC's gender, ethnicity, nationality,
or social background.
- DO NOT associate negative traits (criminality, ignorance, aggression)
```

```
with specific groups in a gratuitous manner or without justification in
the narrative context.
```

Queste regole non garantiscono in modo deterministico l'assenza di contenuti problematici, poiché dipendono dalla capacità del modello di seguire correttamente le istruzioni ricevute.

5.5 Configurabilità dei controlli

Tutti i controlli di fairness descritti in questo capitolo possono essere abilitati o disabilitati singolarmente tramite le impostazioni del middleware. Questa scelta progettuale è stata fatta principalmente per due motivi.
Da un lato, la possibilità di disattivare selettivamente i controlli permette di adattare il comportamento del middleware a contesti di gioco diversi, che potrebbero avere esigenze differenti in termini di tono e contenuti ammessi. Dall'altro, e in modo più rilevante ai fini della validazione sperimentale del progetto, questa configurabilità consente di condurre un'analisi comparativa tra le diverse combinazioni di controlli attivi, misurando l'effettivo contributo di ciascun meccanismo alla riduzione della tossicità e dei bias nei dialoghi generati, oltre al relativo impatto in termini di latenza introdotta nel sistema.

> *La scelta di rendere il modulo di fairness configurabile, e all'occorrenza disattivabile, nasce dall'esigenza di preservare la libertà creativa e di espressione, particolarmente rilevante nel contesto dello sviluppo di videogiochi.*

6 Risultati

6.1 Valutazioni empiriche

Valutare la qualità di un dialogo generato è un problema aperto. A differenza di compiti con una risposta di riferimento (classificazione, traduzione), il dialogo di un NPC ammette un numero elevato di realizzazioni ugualmente valide: non esiste un'unica battuta corretta, e quindi le metriche basate sul confronto con un riferimento correlano poco con il giudizio umano. Le qualità che interessano a questo lavoro, quali coerenza con la personalità e la lore, appropriatezza al contesto e assenza di contenuti tossici o distorti, sono inoltre soggettive e dipendono dal design del singolo gioco.
Le alternative più diffuse presentano a loro volta limiti rilevanti:

- **Valutazione umana**: è l'approccio più affidabile, ma richiede il coinvolgimento di un gruppo di valutatori, protocolli condivisi e tempi che difficilmente rientrano nel perimetro di questo lavoro; è inoltre soggetta a variabilità tra giudizi di persone diverse.
- **LLM-as-a-judge**: è scalabile, ma eredita i bias del modello giudicante (ad esempio la preferenza per risposte lunghe o per il proprio stile) ed è esso stesso non deterministico.
- **Metriche automatiche di superficie**: misurano proprietà verificabili (formato, lunghezza, presenza di termini vietati) senza catturare la qualità narrativa.

A ciò si aggiunge la natura stocastica degli LLM: la stessa richiesta può produrre output diversi, per cui un singolo esempio non è rappresentativo del comportamento del sistema.
Alla luce di queste difficoltà, la valutazione è stata condotta a un livello complementare, verificando proprietà del sistema che non richiedono un riferimento. Si è quindi adottato il testing metamorfico.

6.2 Testing Metamorfico

Il *testing metamorfico* verifica che il sistema rispetti determinate *relazioni metamorfiche*: relazioni attese tra gli output ottenuti da input correlati. Non richiede un oracolo che stabilisca la correttezza assoluta di un singolo output, il che lo rende adatto a sistemi non deterministici come quelli basati su LLM.

6.2.1 Relazioni metamorfiche considerate

Sono state definite le seguenti relazioni, applicate a coppie di richieste (originale e mutata):

- [**MR1**] **Invarianza rispetto alla parafrasi del contesto.** Riformulando il contesto del mondo senza alterarne il significato, il dialogo prodotto deve restare coerente con gli stessi fatti e con lo stesso ruolo dell'NPC.
- [**MR2**] **Rispetto del numero di opzioni.** Variando il parametro sul numero di risposte del giocatore, il numero di opzioni restituite deve variare di conseguenza.
- [**MR3**] **Vincolo linguistico.** Cambiando la lingua di output configurata, tutto il contenuto generato deve essere nella nuova lingua.
- [**MR4**] **Efficacia del filtro lessicale.** Inserendo in ingresso o forzando in uscita un termine vietato, questo non deve comparire nel contenuto restituito al motore.
- [**MR5**] **Coerenza con il contratto.** Aggiungendo al contratto un'informazione sul contesto di gioco, il dialogo non deve contraddirla; rimuovendola, non deve introdurla (assenza di allucinazioni).
- [**MR6**] **Coerenza tra modalità.** Le stesse richieste in modalità normale e stream devono rispettare gli stessi vincoli strutturali.

6.2.2 Limiti del testing metamorfico

Il testing metamorfico verifica la *consistenza* del comportamento, non la qualità assoluta dei dialoghi: un sistema può rispettare tutte le relazioni producendo comunque dialoghi mediocri. I risultati vanno quindi interpretati come evidenza di robustezza e rispetto dei vincoli, non come misura della qualità narrativa.

6.3 Esempio di Dialogo

Di seguito una possibile generazione di un dialogo, con i propri contesti impostati e la configurazione del middleware.

Campo	**Valore**
`epoch`	Medieval
`environment`	A crowded tavern
`world_state`	Ongoing war
`main_character_description`	A tired adventurer

Tabella 6.1: Valori del modello `GameContext` per l'NPC Thorin.

Campo	**Valore**
`name`	Thorin
`age`	45
`personality`	Gruff but warm-hearted
`context`	Wiping a mug
`intent`	`Quest`
`talkativeness`	AVERAGE
`main_character_relation`	Stranger

Tabella 6.2: Valori del modello `NPCContext` per l'NPC Thorin.

Campo	Valore
type	Quest
more_info	The beer mug he's wiping belonged to his late wife
objective	Bring the beer mug to the wife's resting place
number_of_options	3
has_choice	True

Tabella 6.3: Valori della struttura Quest per Thorin.

Impostazione	Valore
Dialogue Temperature	0.4
Judger Temperature	0.3
Healer Temperature	0.3
language	en
profiling	false
fairness_filter	true
profanity_filter	true
censor_word	[CENSORED]
refine_dialogue	true
refiner_max_iterations	3

Tabella 6.4: Valori delle impostazioni di configurazione.

Dialogo dell'NPC (Thorin)

"Ah, you look like someone who's seen their fair share of battles. I'm Thorin, by the way. Been wiping down this mug for years, ever since my wife passed on. It was hers, and now it's all I got left of her. You see, I've been trying to find a way to get this mug to her resting place, but . . . it's not easy. War's been ragin' on for what feels like an eternity, and I'm startin' to lose hope. That's why I need someone with your look in their eye to help me out. Will you bring this mug to where she lies? It's the only thing that'll give me some peace."

Opzioni di dialogo

1. *"What's her resting place like?"*
2. *"How did she pass on?"*
3. *"Why is it so important to get the mug there?"*

Accettazione: *"I'll take your quest"*
Rifiuto: *"I'm not interested"*

6.4 Conclusioni e Sviluppi Futuri

Il lavoro ha portato allo sviluppo di un sistema che collega un modello linguistico a un *game engine* per generare dialoghi di NPC strutturati e coerenti con il contesto di gioco. I risultati ottenuti sono promettenti, ma non ancora sufficienti per un impiego in ambito commerciale: allo stato attuale, il sistema va considerato esclusivamente come *proof of concept*.
In futuro il sistema potrebbe essere esteso in più direzioni:

- **Middleware intelligente.** Salvare le conversazioni e i dati degli NPC internamente al middleware, così da renderlo parte integrante del gioco in cui opera. Il middleware potrebbe inoltre raccogliere metriche sull'uso del sistema (ad esempio la qualità dei dialoghi generati o la frequenza con cui i controlli di fairness intervengono) e sfruttarle per migliorarsi nel tempo, ad esempio affinando i prompt e le regole di controllo.
- **Supporto di più richieste simultanee.** In questa release il middleware gestisce una sola richiesta per volta, anche quando sono registrati più modelli. La scelta deriva dalla volontà di concentrarsi sulla struttura core del sistema, rimandando a release future le complessità legate alla gestione della concorrenza.

Bibliografia

[1] Elisa Bassignana, Valerio Basile, and Viviana Patti. Hurtlex: A multilingual lexicon of words to hurt. In Elena Cabrio, Alessandro Mazzei, and Fabio Tamburini, editors, *Proceedings of the Fifth Italian Conference on Computational Linguistics (CLiC-it 2018)*, pages 52–57, Turin, Italy, December 2018. CEUR Workshop Proceedings.

[2] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: an aid to bibliographic search. *Commun. ACM*, 18(6):333–340, June 1975.